EE2002-Tutorial 5 Solutions

1. Design an Excess-3 to Binary Converter. Clearly explain the design steps and provide the necessary logic equations.Take unused cases as don't cares

Note:Excess-3 (XS-3) code is a self-complementary binary-coded decimal (BCD) system in which each decimal digit (0-9) is represented by its binary equivalent plus 3.

Solution:

Step 1: Excess-3 (XS-3) is a self-complementary binary-coded decimal (BCD) system where each decimal digit (0-9) is represented by its binary equivalent plus 3.

$$\text{XS-3 Code} = \text{BCD} + 3$$

Step 2: Truth Table

Excess-3 Input (X3 X2 X1 X0)	Binary Output (B3 B2 B1 B0)	Don't Care
0011 (3)	0000 (0)	No
0100 (4)	0001 (1)	No
0101 (5)	0010 (2)	No
0110 (6)	0011 (3)	No
0111 (7)	0100 (4)	No
1000 (8)	0101 (5)	No
1001 (9)	0110 (6)	No
1010 (10)	0111 (7)	No
1011 (11)	1000 (8)	No
1100 (12)	1001 (9)	No

Table 1: Truth Table for Excess-3 to Binary Decoder

Step 2: Boolean Equations using Sum of Minterms and Don't Care Conditions Using Karnaugh maps, we derive the following expressions:

$$B3 = \sum m(11, 12) + d(0, 1, 2, 13, 14, 15)$$
$$B2 = \sum m(7, 8, 9, 10) + d(0, 1, 2, 13, 14, 15)$$
$$B1 = \sum m(5, 6, 9, 10) + d(0, 1, 2, 13, 14, 15)$$
$$B0 = \sum m(4, 6, 8, 10, 12) + d(0, 1, 2, 13, 14, 15)$$

After simplifying the Boolean expressions using Karnaugh maps, we obtain the following minimized forms:

Below are the Karnaugh maps for each Boolean function:

X1X0 / X3X2	00	01	11	10
00	X	X	0	X
01	0	0	0	0
11	1	X	X	X
10	0	0	1	0

Result

$F = X3\,X2 + X3\,X1\,X0$

(a) Karnaugh Map for B3

X1X0 / X3X2	00	01	11	10
00	X	X	0	X
01	0	0	1	0
11	0	X	X	X
10	1	1	0	1

Result

$F = \overline{X2}\,\overline{X0} + \overline{X2}\,\overline{X1} + X2\,X1\,X0$

(b) Karnaugh Map for B2

X1X0 / X3X2	00	01	11	10
00	X	X	0	X
01	0	1	0	1
11	0	X	X	X
10	0	1	0	1

Result

$F = X1\,\overline{X0} + \overline{X1}\,X0$

(a) Karnaugh Map for B1

X1X0 / X3X2	00	01	11	10
00	X	X	0	X
01	1	0	0	1
11	1	X	X	X
10	1	0	0	1

Result

$F = \overline{X0}$

(b) Karnaugh Map for B0

Implementation of the Excess-3 to Binary circuit as shown below

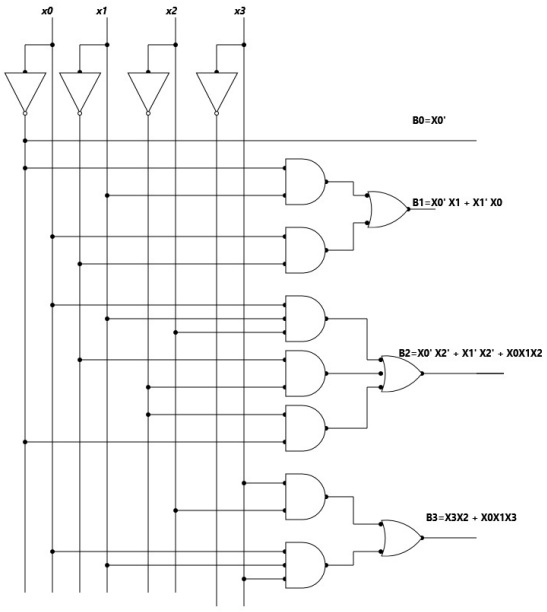


Figure 3: Excess-3 to Binary Converter

2. Design a 4-to-2 priority encoder with inputs D_0, D_1, D_2 and D_3 such that D_3 has the highest priority followed by D_2 and so on. Draw the appropriate truth table and develop the logic equations for the two outputs in terms of the inputs. You may ignore the third output (whose purpose is to resolve the ambiguity relating to all inputs being 0). Incorporate also the various don't care cases in your solution and perform simplification of the expressions for the outputs.

Solution:

Step 1: A 4-to-2 priority encoder takes four input lines I_3, I_2, I_1, I_0 and encodes them into a 2-bit output Y_1, Y_0.

- If multiple inputs are HIGH (1), the highest-numbered input takes precedence.
- If no inputs are HIGH, the output is undefined (XX, indicating a don't-care condition).

Step 2: Constructing the Truth Table

Inputs $I_3I_2I_1I_0$	Outputs Y_1Y_0
0000	XX
0001	00
001X	01
01XX	10
1XXX	11

Don't Care Conditions:

- When no input is HIGH (0000), the output is defined (XX).
- When a higher-priority input is HIGH, lower-priority inputs are ignored (represented by X).

Step 3: Karnaugh Maps for Output Expressions Now, we can draw K-maps for Y_0 and Y_1 based on the above truth table as shown below.



(a) Karnaugh Map for Y_0



(b) Karnaugh Map for Y_1

Implementation circuit of the 4:2 priority encoder as shown below

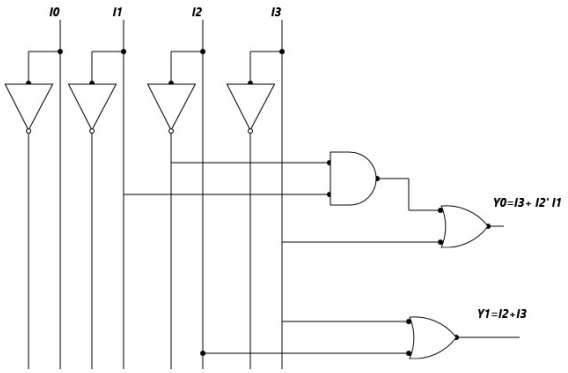


Figure 5: 4:2 Priority encoder

3. Consider the multiplexer as shown in Figure 6, where I_0, I_1, I_2, I_3 are four data input lines that can be selected using the two address line combinations $A_1A_0 = 00, 01, 10, 11$ respectively, and EN is the enable input. Find the expression for f which is implemented by the circuit shown in the figure.

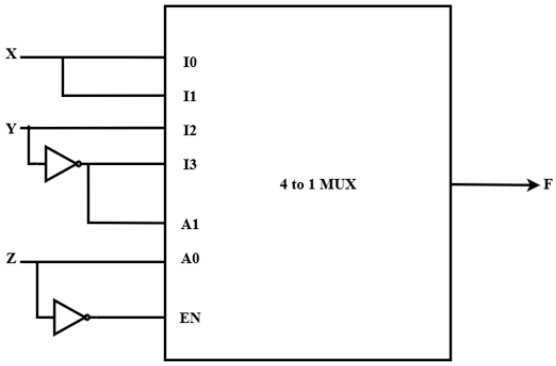


Figure 6: 4-to-1 Multiplexer (MUX)

Solution:

The output of the circuit is given by $f(x, y, z)$. When $Z = 0$, only the MUX is enabled.

A_1	A_0	f	
0	0	x	(selected)
0	1	0	(disabled)
1	0	y	(selected)
1	1	0	(disabled)

The Boolean expression for f can be derived as:

$$f = \bar{A}_1\bar{A}_0x + A_1\bar{A}_0y$$

Since $A_1 = \bar{y}$ and $A_0 = z$, we substitute:

$$f = y\bar{z}x + \bar{y}\bar{z}y$$

$$f = xy\bar{z}$$

Thus, the final expression for f is:

$$f = xy\bar{z}$$

4. Realize a 16-to-1 MUX using only 2-to-1 MUXes. Draw the circuit.

Solution:

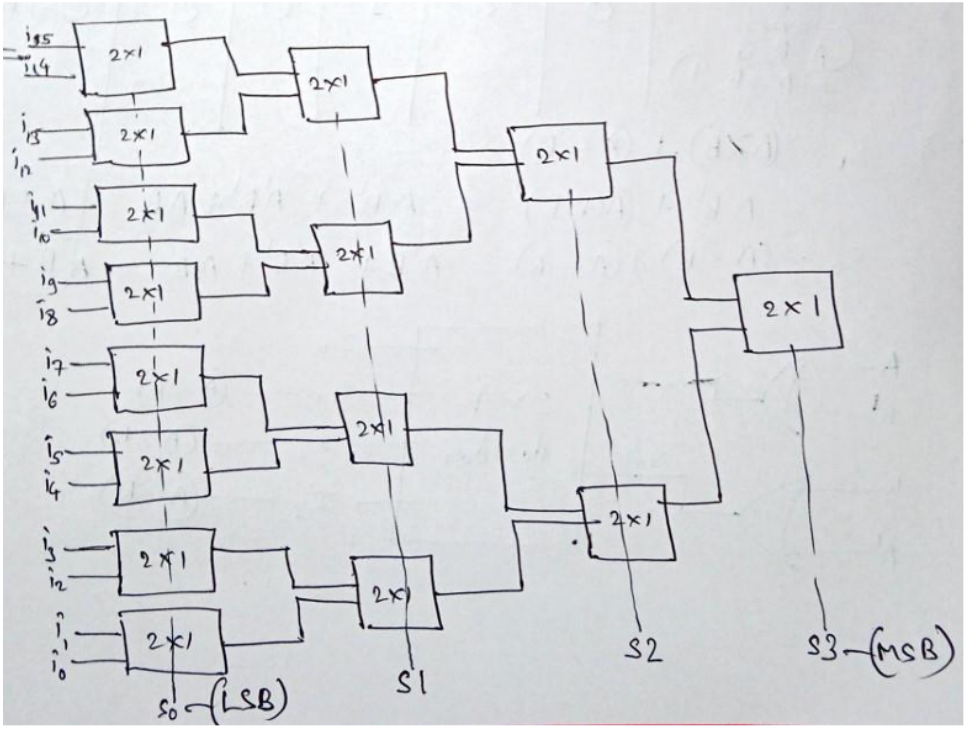


Figure 7: 4-bit Adder/Subtractor Circuit

- First Level: Uses **8 MUXes**, each taking **2 inputs** and controlled by $\mathbf{S_0}$.
- Second Level: Uses **4 MUXes**, each taking **2 outputs** from the first level, controlled by $\mathbf{S_1}$.
- Third Level: Uses **2 MUXes**, each taking **2 outputs** from the second level, controlled by $\mathbf{S_2}$.
- Final Level: Uses **1 MUX** to generate the **final output**, controlled by $\mathbf{S_3}$.

5. Consider the 3-input majority function. Design the logic circuit for this using (a) 8:1 Mux and (b) 4:1 Mux. Choose A(MSB),B and C as inputs.

Solution:

Figure 8 gives the solution.

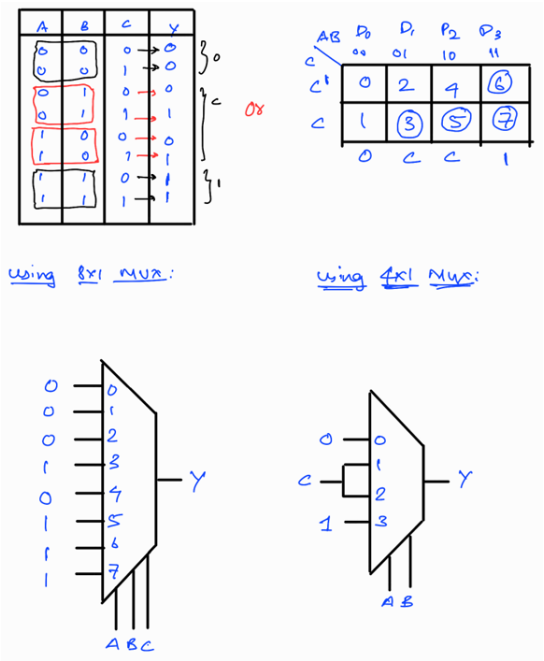


Figure 8: Majority circuit using 8:1 and 4:1 MUX

6. We have already seen the role of Gray code in Karnaugh maps. In this question, we will look at a conversion task involving 4-bit binary and Gray codes. Consider Table 2 where the input (binary) is given by $abcd$ while the corresponding output, denoted by $g_3g_2g_1g_0$, is in Gray code format. Implement the output g_1 using NOR gates and outputs g_2 and g_0 using NAND gates.

a	b	c	d	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Table 2: Truth Table for Inputs and Outputs

Solution:

From the truth table, the Boolean expressions for each Gray code output in sum of minterms notation are:

$$g_3 = \sum m(8, 9, 10, 11, 12, 13, 14, 15)$$

$$g_2 = \sum m(4, 5, 6, 7, 12, 13, 14, 15)$$

$$g_1 = \sum m(2, 3, 6, 7, 10, 11, 14, 15)$$

$$g_0 = \sum m(1, 3, 5, 7, 9, 11, 13, 15)$$

The Boolean expressions for Gray code conversion are derived and visualized in the following figure:

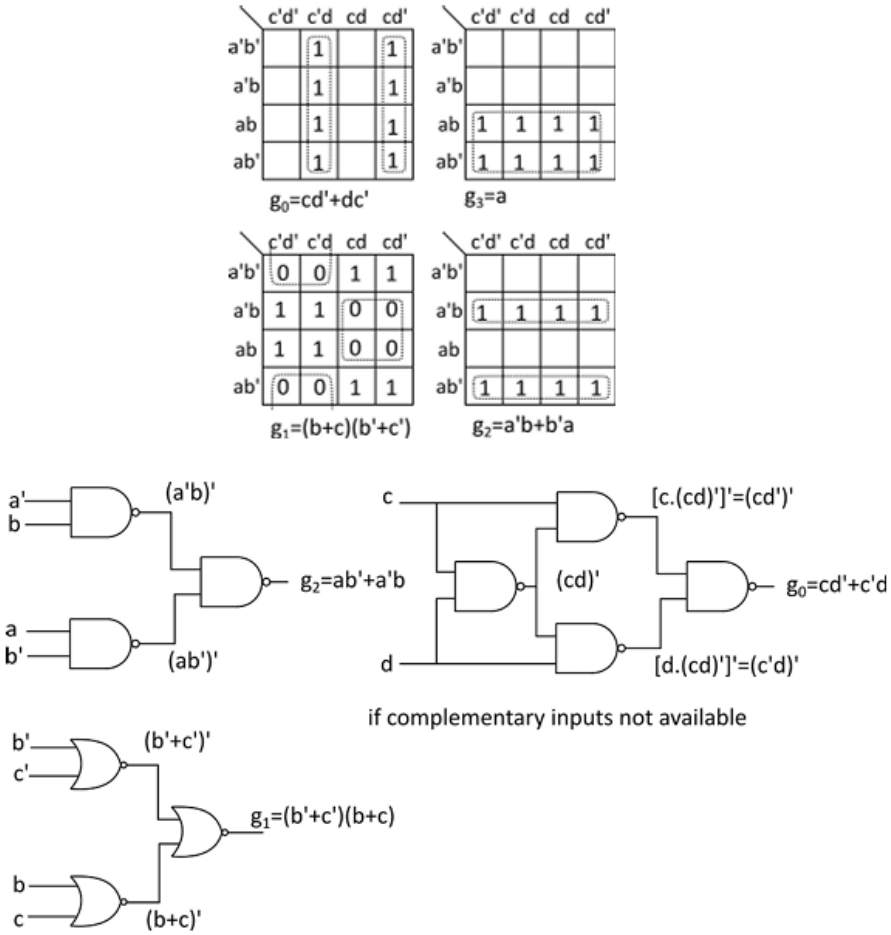


Figure 9: Derived Boolean Expressions

7. Implement the function $F(A, B, C, D) = \sum(1, 2, 3, 4, 7, 8, 9, 10, 12, 13, 15)$ using:

(a) An 8:1 MUX and required logic gates.

(b) A 4:1 MUX and required logic gates.

Draw the implementation tables for both cases.

Solution:

Implementation Table

The implementaion table is used to map the values of the inputs to the corresponding output for the given logic function. In the case of a MUX implementation, the implementation table shows how the select lines A, B, C (or other inputs, depending on the MUX) influence the function's result and help determine the necessary input conditions for the MUX. This table is essential for designing the logic required to implement the function with a MUX.

Part (a): 8:1 MUX Implementation

The Boolean function $F(A, B, C, D) = \sum(1, 2, 3, 4, 7, 8, 9, 10, 12, 13, 15)$ can be implemented using an 8:1 multiplexer (MUX) with A, B, C as the select lines. Now, for the 8:1 MUX, we will use A, B, C as the select lines and assign the corresponding values of F for each of the 8 inputs based on the truth table.

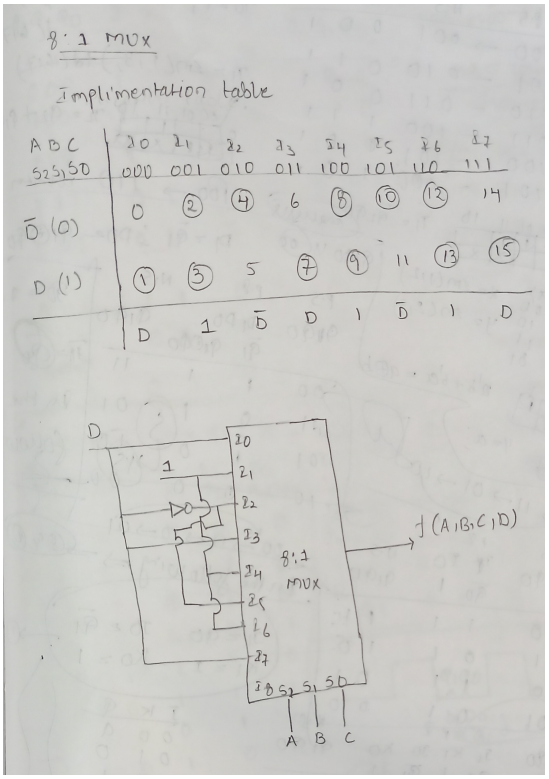


Figure 10: 8:1 MUX Implementation

Part (b): 4:1 MUX Implementation

For the 4:1 MUX implementation, we will use A and B as the select lines and construct the MUX using logic for the variables C and D.

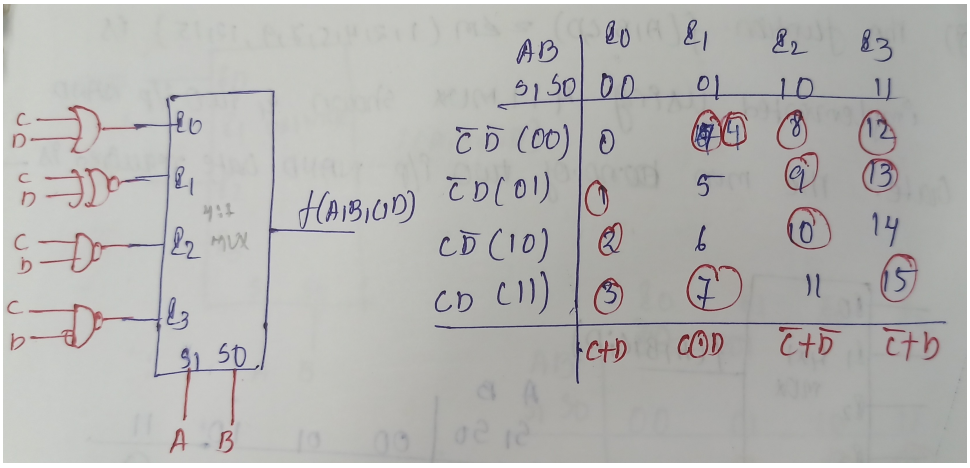
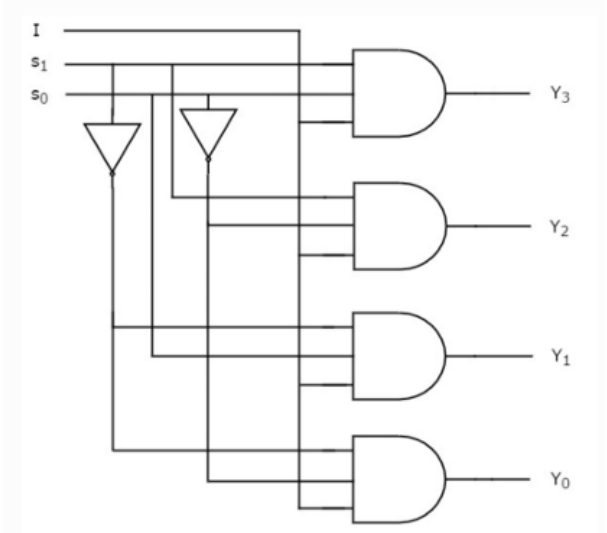


Figure 11: 4:1 MUX Implementation

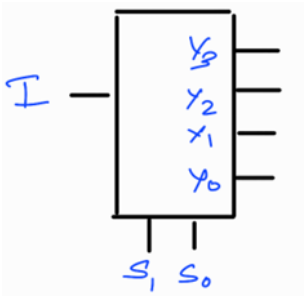
8.Construct the truth table and output expressions Y_0, Y_1, Y_2, Y_3 for the combinational circuit provided below, with input I and S_1 (MSB), S_0 as select lines. Comment on its operation by comparing it with a multiplexer.



Solution:

Output Expressions

$$Y_3 = S_1 S_0 I$$
$$Y_2 = I S_1 \overline{S_0}$$
$$Y_1 = I \overline{S_1} S_0$$
$$Y_0 = I \overline{S_1 S_0}$$



(a) 1:4 DeMUX Implementation

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

(b) Truth Table

Figure 12: 1:4 MUX and Corresponding Truth Table

A 1:4 demultiplexer takes a single input I and routes it to one of four outputs based on the select lines S_1 and S_0.

- Only one output is active (high) at a time, depending on the select lines.
- Unlike a multiplexer (which selects one input among many to pass to a single output), a demultiplexer distributes one input to multiple outputs.
- The output expressions confirm that the input I is directed to the appropriate Y_n output based on the select signals.